

DATA ANALYTICS

IT ACADEMY | BCN ACTIVA

SPRINT 09

Consultas con MongoDB
15 de julio de 2025



Letiane Benincá
benincalf@gmail.com

IT ACADEMY



TAREA S9.01

CONSULTAS CON MONGODB.

Objetivos:

- Crear una base de datos en MongoDB con colecciones dadas.
- Realizar consultas para extraer información relevante de cada colección.
- Evaluar la capacidad analítica para trabajar con datos en MongoDB.

Duración: 3 días.

Entrega: Almacena en un repositorio de tu GitHub una carpeta que contenga:

- Un PDF que incluya una captura de pantalla de MongoDB Compass donde se pueda observar cada script de las consultas que has realizado y el resultado obtenido para cada ejercicio.

En la entrega, coloca el enlace a este repositorio.

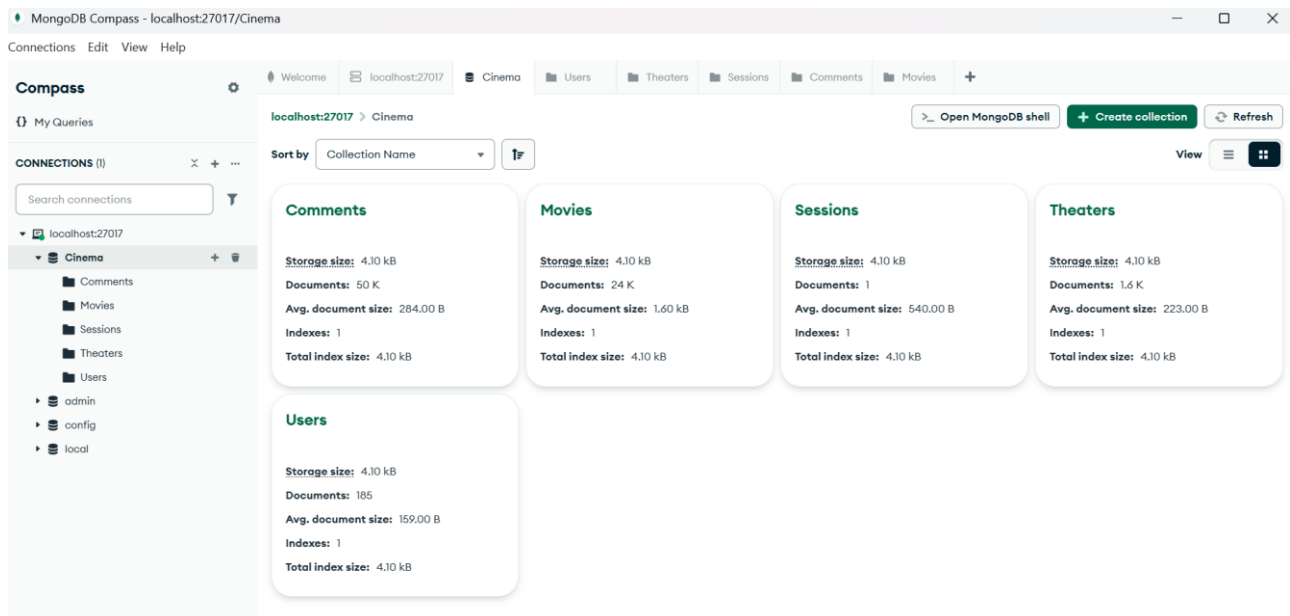
DESCRIPCIÓN

Trabajaremos con una base de datos que contiene colecciones relacionadas con una aplicación de entretenimiento cinematográfico:

- **users:** Almacena información de usuarios/as, incluyendo nombres, emails y contraseñas cifradas.
- **theatres:** Contiene datos de cines, como ID, ubicación (dirección y coordenadas geográficas).
- **sessions:** Guarda sesiones de usuario, incluyendo ID de usuario y tokens JWT para la autenticación.
- **movies:** Incluye detalles de películas, como sinopsis, géneros, duración, reparto, comentarios, año de estreno, directores, clasificación y premios.
- **comments:** Almacena comentarios de usuarios/as sobre películas, con información del autor/a del comentario, ID de la película, texto del comentario y la fecha.

Realizarás algunas consultas solicitadas por el/la cliente, quien está evaluando si serás capaz o no de encargarte de la parte analítica del proyecto vinculado a su base de datos.

Inicialmente, cargamos los datos en MongoDB Compass:



NIVEL 1

1. Ejercicio 01:

- Muestra los 2 primeros comentarios que hay en la base de datos.

```
>_MONGOSH
> use Cinema
< switched to db Cinema
> db.Comments.find().limit(2)
< {
  _id: ObjectId('5a9427648b0beebe69579cc'),
  name: 'Andrea Le',
  email: 'andrea_le@fakegmail.com',
  movie_id: ObjectId('573a1390f29313caabcd418c'),
  text: 'Rem officii eaque repellendus amet eos doloribus. Porro dolor voluptatum voluptates neque culpa molestias. Voluptate unde nulla temporibus ul',
  date: 2012-03-26T23:20:16.000Z
}
{
  _id: ObjectId('5a9427648b0beebe69579cf'),
  name: 'Greg Powell',
  email: 'greg_powell@fakegmail.com',
  movie_id: ObjectId('573a1390f29313caabcd41b1'),
  text: 'Tenetur dolorum molestiae ea. Eligendi praesentium unde quod porro. Commodi nisi sit placeat rerum vero cupiditate neque. Dolorum nihil vero a',
  date: 1987-02-10T00:29:36.000Z
}
```

La consulta presenta los dos primeros comentarios (Comments), que son de Andrea Le e Greg Powell. Utilizando el find y filtrando por solo los 2 primeros (limit(2)).

- ¿Cuántos usuarios/as tenemos registrados?

```
> db.users.find().count()
< 0
> db.Users.find().count()
< 185
```

Como en una consulta de SQL, hacemos la agrupación con el `find` para definir usuarios y con `count` para contarlos. Tenemos 185 usuarios en total. Aquí evidenciamos a la sensibilidad a los caracteres (user en minúsculos no hay registro y luego, ajustando a la db con el U mayúsculo, si que se ha ajustado).

- ¿Cuántos cines hay en el estado de California?

```
> db.Theaters.find({ "location.address.state": "CA" }).count()
< 169
```

Este código realiza una consulta en la colección **Theaters** para encontrar todos los documentos donde el estado (`location.address.state`) es California ("CA") y luego cuenta (método `count()`) cuántos documentos cumplen con ese criterio. El resultado indica que hay 169 teatros ubicados en California.

- ¿Quién fue el/la primer/a usuario/a en registrarse?

```
> db.Users.find().sort({_id:1}).limit(1)
< {
  _id: ObjectId('59b99db4cfa9a34dcd7885b6'),
  name: 'Ned Stark',
  email: 'sean_bean@gameofthron.es',
  password: '$2b$12$UREFwsRUoyF0CRqGNK0Lz00HM/jLhgUCNNIJ9RJAqMUQ74cr1J1Vu'
}
```

- Realizamos una consulta en la colección `Users`, buscando todos los documentos (`find()`), ordenando por el campo `_id` (`sort({_id:1})`) en orden ascendente (1 = ascendente y -1 = descendente) y limitando (`limit(1)`) por el primer registro. El resultado muestra el primer documento de la colección `Users`, que corresponde al usuario "Ned Stark".

- ¿Cuántas películas de comedia hay en nuestra base de datos?

```
> db.Movies.find({$and:[{type:"movie"},{genres:"Comedy"}]}).count()
< 7002
```

Utilizamos la colección Movies, que contiene los documentos relacionados a las películas, con el método find(), aplicamos filtros y condiciones para la busca. El operador \$and permite especificar múltiples condiciones que deben cumplirse al mismo tiempo, en esta consulta {type: "movie"} y {genres: "Comedy"}. Para que este documento sea seleccionado, debe cumplir ambas condiciones, movie y Comedy. Por último, aplicamos el método count() para contar los resultados, llegando al valor de 7.002 películas de la categoría Comedy.

2. Ejercicio 02: Muéstrame todos los documentos de las películas producidas en 1932, pero cuyo género sea drama o estén en francés.

```
> db.Movies.find({$and:[{type:"movie"},{year:1932},{$or:[{genres:"Drama"},{languages:"French"}]}]}).count()
< 18
```

En esta consulta, tenemos que utilizar dos operadores lógicos (\$and y \$or) para buscar documentos que cumplan estas condiciones. Se da una condición principal con \$and [], que permite especificar múltiples condiciones que deben cumplirse al mismo tiempo, en este caso, type: movies; year: 1932 y con el operador \$or (que sería una condición interna), especificamos una condición alternativa. Un documento será seleccionado si cumple al menos una de estas condiciones: genre: Drama y languages: French. Verificamos que tenemos 18 películas que cumplen todas las condiciones.

```
> db.Movies.find({$and:[{type:"movie"},{year:1932},{$or:[{genres:"Drama"},{languages:"French"}]}]}
< {
  _id: ObjectId('573a1391f29313caabcd9458'),
  plot: 'A young artist draws a face at a canvas on his easel. Suddenly the mouth on the drawing comes into life and starts talking. The artist tries t
  runtime: 55,
  rated: 'UNRATED',
  cast: [
    'Enrique Rivero',
    'Elizabeth Lee Miller',
    'Pauline Carton',
    'Odette Talazac'
  ],
  num_mflix_comments: 1,
  poster: 'https://m.media-amazon.com/images/M/MV5BYWY3ODE5ZWEtYjlmVi00NjA4LTk4ZWYtMzBhZDE5MjY0YTUxXkEyXkFqcGdeQXVyNzI4MDMyMTU@._V1_SV1000_SX677_AL_.jp
  title: 'The Blood of a Poet',
  lastupdated: '2015-09-16 13:13:05.537000000',
  languages: [
    'French'
  ],
  released: 2010-05-20T00:00:00.000Z,
  directors: [
    'Jean Cocteau'
  ],
  writers: [
    'Jean Cocteau'
```

En esta consulta, sin el count, se puede verificar los datos de las 18 películas francesas.

3. **Ejercicio 03:** Muéstrame todos los documentos de películas estadounidenses que tengan entre 5 y 9 premios y que fueron producidas entre 2012 y 2014.

```
> db.Movies.countDocuments({
  countries: "USA",
  "awards.wins": { $gte: 5, $lte: 9 },
  year: { $gte: 2012, $lte: 2014 }
})
< 166
```

Se utiliza el método `countDocuments()` para contar documentos en la colección `Movies` que cumplen con varias condiciones específicas. Los filtros se aplican al mismo tiempo, siendo que el resultado solo se presentará si cumple todas las condiciones: `countries: USA`; `awards.wins` entre 5 y 9 (incluido) e `year` entre 2012 y 2014 (incluido). Para lograrlo, utilizamos operadores de comparación para filtrar documentos según un rango de valores. Siendo `$gte`: greater than or equal to (mayor o igual que $\geq$) y `$lte`: less than or equal to (menor o igual que $\leq$). En este caso tenemos 166 películas que cumplen todas las condiciones.

```
> db.Movies.find({countries: "USA", "awards.wins": {$gte:5,$lte:9}, year:{$gte:2012,$lte: 2014}})
< {
  _id: ObjectId('573a13acf29313caabd29366'),
  fullplot: "The manager of the negative assets sector of Life magazine, Walter Mitty, has been working for sixteen years for the magazine and has a te
  imdb: {
    rating: 7.4,
    votes: 211230,
    id: 359950
  },
  year: 2013,
  plot: "When his job along with that of his co-worker are threatened, Walter takes action in the real world embarking on a global journey that turns i
  genres: [
    'Adventure',
    'Comedy',
    'Drama'
  ],
  rated: 'PG',
  metacritic: 54,
  title: 'The Secret Life of Walter Mitty',
  lastupdated: '2015-08-31 00:10:51.747000000',
  languages: [
    'English',
    'Spanish',
```

Aquí, tenemos el listado de las películas que cumplen las condiciones filtradas.

NIVEL 2

- Ejercicio 01:** Cuenta cuántos comentarios escribe un/a usuario/a que utiliza "GAMEOFTHRON.ES" como dominio de correo electrónico.

```
> db.Comments.find({email: {$regex: /@gameofthron.es$/i}}).count()
< 0
> db.Comments.find({email: {$regex: /@gameofthron.es$/i}}).count()
< 22841
```

Utilizamos el operador \$regex para buscar documentos basados en un patrón de texto. El filtro se usa condicionando el campo email, donde queremos encontrar y contar los emails con el texto 'gameofthron.es'. Para esto, se utilizó la expresión regular: /@gameofthron.es\$/i, donde los // indica que esta dentro de una expresión regular. El '@gameofthron.es' es el texto por buscar (usamos @, porque pide el dominio del correo). El \$ significa que debe coincidir al final del campo del email y el i para identificar que la busca debe ser case-insensitive, sin distinguir mayúsculas y minúsculas. Encontramos un total de 22.841 comentarios con este dominio de correo.

- Ejercicio 02:** ¿Cuántos cines hay en cada código postal dentro del estado Washington D. C. (DC)?

```
> db.Theaters.aggregate([{$match: {'location.address.state': 'DC'}}, {$group: {_id: '$location.address.zipcode', numero_cinemas: {$count: {}}}}])
< [
  {
    _id: '20016',
    numero_cinemas: 1
  },
  {
    _id: '20010',
    numero_cinemas: 1
  },
  {
    _id: '20002',
    numero_cinemas: 1
  }
]
```

Utilizamos el método aggregate() para realizar una operación de agregación en la colección Theaters. El operador \$match filtra los documentos conforme las condiciones: {'location.address.state': 'DC'}, solo se considerarán los teatros ubicados en el estado de DC. El operador \$group, agrupa los documentos por un campo y permite usar operaciones de agregación, en este caso: _id: '\$location.address.zipcode'. Define el campo por el cual se agrupan los documentos. Por último, calcula el número de documentos en cada grupo, con el método \$count(). El resultado de la consulta es una lista de documentos agrupados por zipcode, junto con el número de cines (numero_cinemas) en cada grupo.

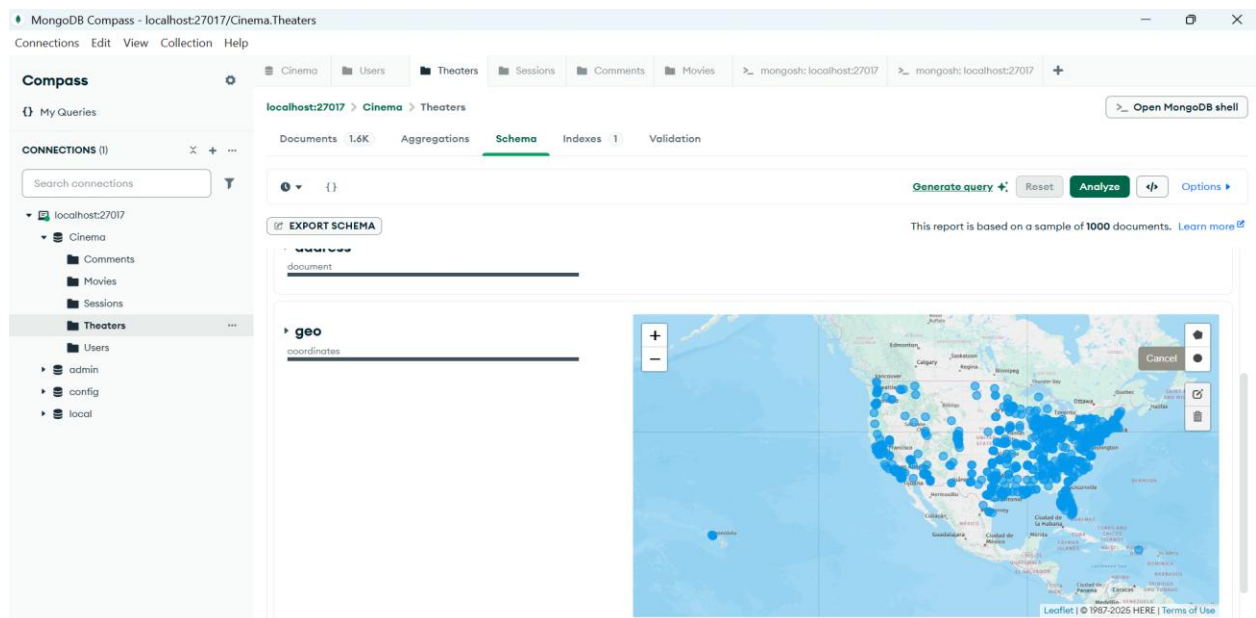
NIVEL 3

- Ejercicio 01:** Encuentra todas las películas dirigidas por John Landis con una puntuación IMDb (Internet Movie Database) entre 7,5 y 8.

```
> db.Movies.find({'type': 'movie', 'directors': 'John Landis', 'imdb.rating': {'$gte': 7.5, '$lte': 8}}).count()
< 4
```

Esta consulta es similar al ejercicio 03, pero un poco más compleja, donde todos los filtros están combinados y se aplican al mismo tiempo, donde el resultado tiene que atender a todas las condiciones: type: movie, directors: 'John Landis', imdb.rating debe estar entre 7.5 y 8 (incluido). Después de aplicar el filtro con find(), se utiliza el método count() para contar cuántos documentos cumplen con todas las condiciones especificadas. El resultado es 4, siendo las películas que cumplen todas las condiciones mencionadas.

- Ejercicio 02:** Muestra en un mapa la ubicación de todos los cines de la base de datos.



En este caso, utilizamos la colección Theaters, vamos a la opción 'Schema' y al final de la página, tenemos el geo, que en función de las coordenadas geográficas (latitud y longitud) de cada teatro. Se crea un mapa interactivo, con la distribución geográfica de los cines en Estados Unidos.